

使用 Gemini 3 和 ADK 建構自己的「討價還價店主」代理

來源：<https://codelabs.developers.google.com/agent-app-gemini-3-adk?hl=zh-tw>

步驟數：12

目錄

1. 總覽
2. 設定 Dukaan (Cloud 專案)
3. 建構攤位 (環境設定)
4. 認識你的助理 (Gemini CLI)
5. 上架 (專案設定)
6. 訓練 Raju (角色)
7. 庫存 (新增工具)
8. Raju's Shop Goes Live (Deploying the Backend)
9. 裝飾商店 (前端 UI)
10. 盛大開幕 (部署前端)
11. 結尾時間 (結論)
12. 清除所用資源

步驟 1 · 約 2 分鐘

1. 總覽

在本程式碼研究室中，我們將建構有趣的內容。想像一下，在數位市集裡，AI 代理是附近最聰明、最風趣的店主。

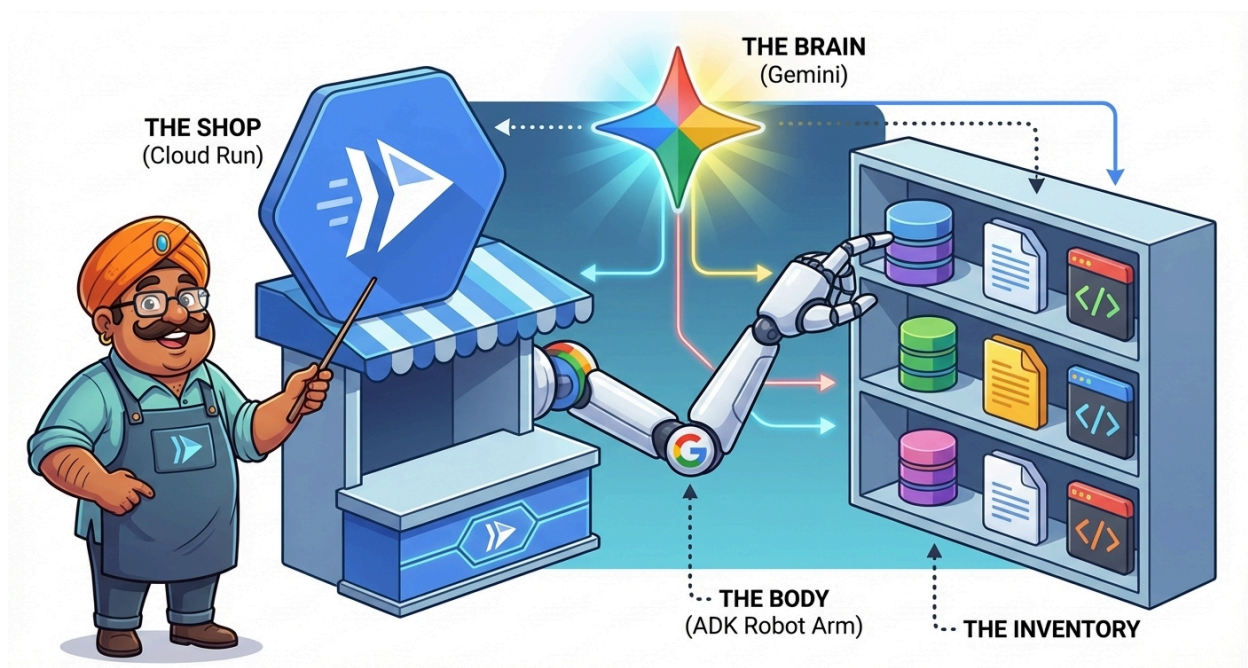
認識全新 AI 店員 **Raju**。他販售的數位文物非常精采，但你必須與他討價還價才能買到！

我們將使用 **Gemini 3** (大腦)、**ADK** (Agent Development Kit - 身體) 和 **Google Cloud Run** (商店位置) 讓 Raju 活起來。



學習目標

我們將為現代化的代理程式應用程式建構整個堆疊：



- 商店 (Cloud Run)：應用程式在網際網路上的位置。
- 大腦 (Gemini 3)：為 Raju 提供智慧功能。
- 身體 (ADK)：連結大腦和工具的架構。

- **庫存 (工具)**：Raju 用來檢查庫存的 Python 程式碼。
- **店面 (UI)**：顧客看到的介面。

您將學習下列內容：

- 設定「Digital Dukaan」(Google Cloud 環境)。
- 「聘用」Raju (取得 Gemini 3 API 金鑰)。
- 教導 Raju 如何討價還價 (系統指令和角色)。
- 為 Raju 提供庫存 (ADK 工具)。
- 建構店面 (前端 UI)。
- 開始營業 (部署至 Cloud Run)。

需求條件

- Google Cloud 專案。
 - 幽默感 (用於討價還價)。
 - Python 基礎知識。
-

步驟 2 · 約 10 分鐘

2. 設定 Dukaan (Cloud 專案)

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

在建構 Raju 之前，我們需要商店位置。在雲端中，這表示專案必須設有帳單帳戶。

步驟 1：啟用帳單



使用本程式碼研究室頂端的橫幅，取得可用於本教學課程的試用帳單帳戶。如果已連結帳單帳戶，可以略過這個步驟。

步驟 2：建立專案



1. 前往「[建立專案](#)」頁面。
2. 輸入不重複的專案名稱 (例如 `raju-shop-agent`)。
3. 在「帳單帳戶」下拉式選單中，選取「試用帳單帳戶」。
4. 按一下「建立」，然後等待專案準備就緒。

步驟 3：驗證帳單連結



1. 前往[連結的帳單帳戶](#)頁面。
2. 如果看到「連結帳單帳戶」選項，請按一下該選項。
3. 確認已選取「試用帳單帳戶」。
4. 現在可以開始建構了！

摘要

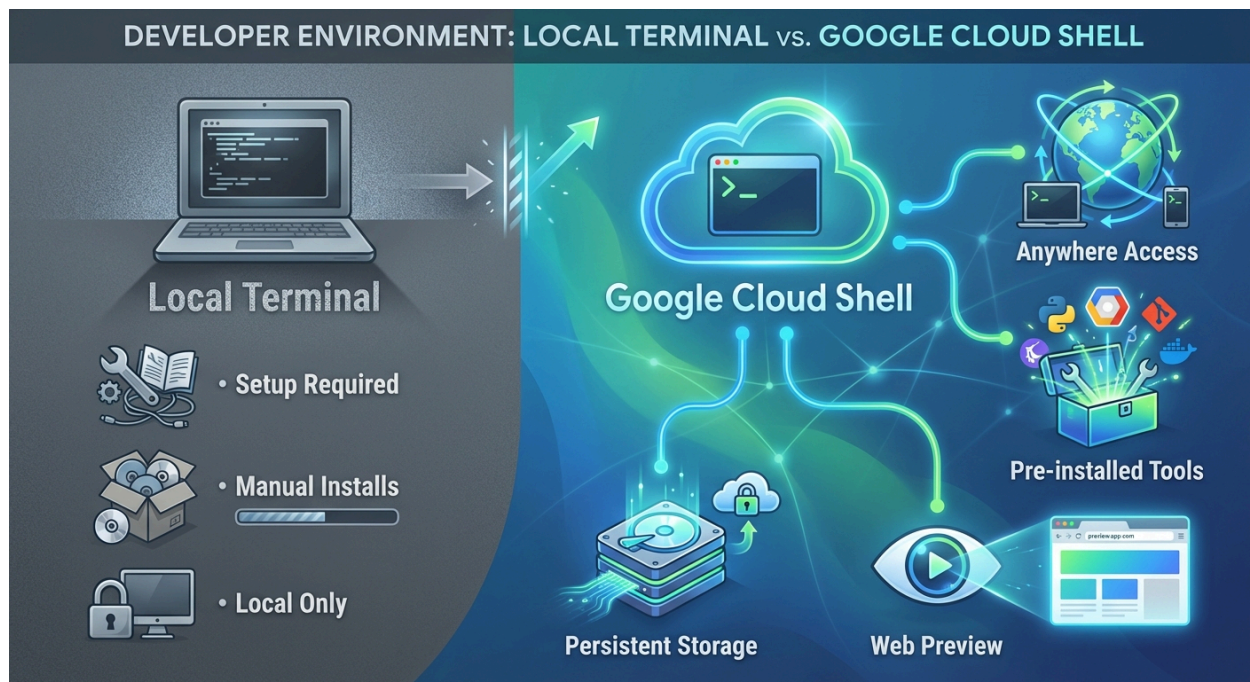
在這個步驟中，您要設定 Google Cloud 專案和帳單帳戶，為應用程式建立基礎。

接著，我們將設定工作區，您會在「準備環境」中建構程式碼。

步驟 3 · 約 5 分鐘

3. 建構攤位 (環境設定)

您需要執行程式碼的位置。您有以下兩種選擇：

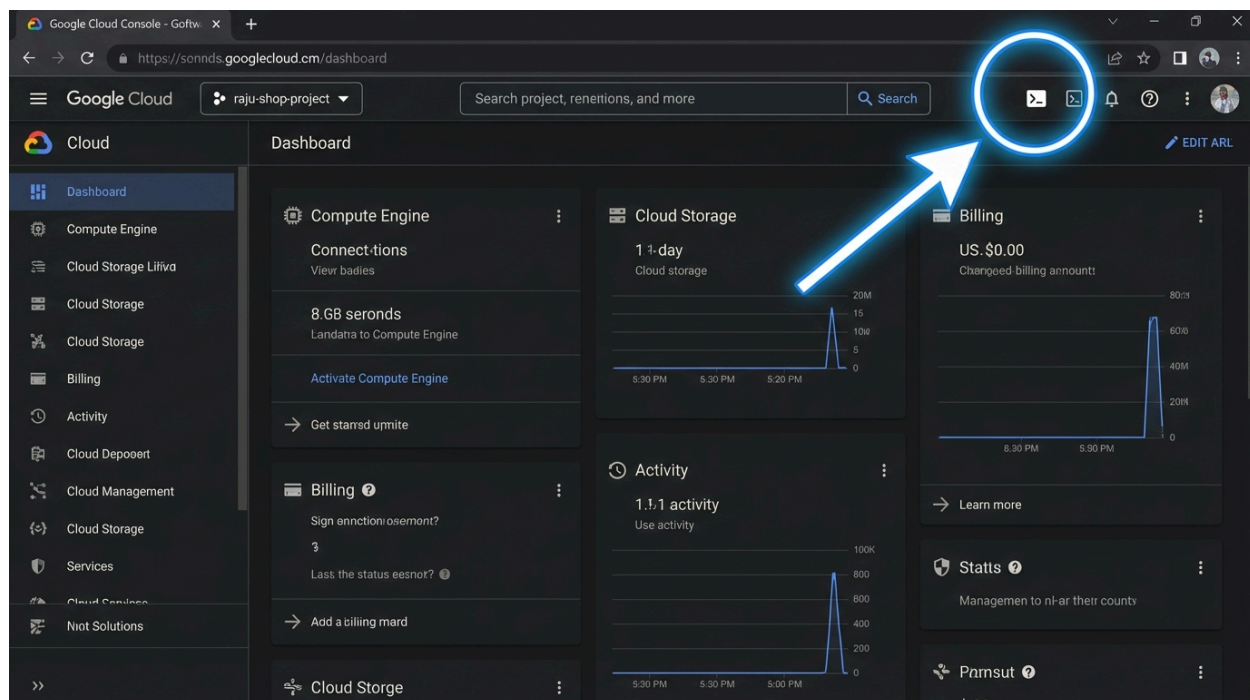


選項 1：Google Cloud Shell (建議)

Cloud Shell 是以瀏覽器為基礎的終端機，已預先安裝所有必要工具 ([Python](#)、[gcloud CLI](#)、[git](#))。這項服務提供永久儲存空間，隨時隨地都能使用。

1. 啟用 Cloud Shell

按一下 Google Cloud 控制台標題右上方的「啟用 Cloud Shell」圖示 (終端機符號)。



Cloud Shell 現在會初始化。學習內容如下：

- **授權**：如果系統提示，請按一下「Authorize」(授權)，允許 Cloud Shell 代表您呼叫 Google Cloud API。
- **佈建**：Cloud Shell 會為工作階段佈建暫時的虛擬機器。
- **連線中**：接著連線至這個 VM。整項程序可能需要一兩分鐘。

等待直到看到命令提示字元 (`user@cloudshell:~ $`)。

2. 驗證專案設定

Cloud Shell 通常會自動選取目前的專案。為確保安全，請驗證：

```
gcloud config get-value project
```

如果沒有傳回專案 ID (`raju-shop-agent`)，請手動設定：

```
gcloud config set project raju-shop-agent
```

選項 2：本機終端機

如果偏好使用自己的筆電終端機 (iTerm、PowerShell 等)，則需要進行更多設定。

1. 安裝必要條件

請確認您已安裝下列項目：

- [Python 3.10 以上版本](#)
- [Google Cloud SDK \(gcloud CLI\)](#)

2. 登入並設定 gcloud

初始化 Google Cloud CLI 並登入帳戶：

```
gcloud auth login
```

設定專案 (將 `raju-shop-agent` 換成實際專案 ID)：

```
gcloud config set project raju-shop-agent
```

設定應用程式預設憑證 (這有助於本機工具與 Google Cloud 通訊)：

```
gcloud auth application-default login
```

摘要

在這個步驟中，您選擇了開發環境 (Cloud Shell 或本機)，並進行設定。

接下來，我們將認識有助於加快建構速度的工具 **Meeting Gemini CLI**。

步驟 4 · 約 5 分鐘

4. 認識你的助理 (Gemini CLI)

現在，隆重介紹 AI 開發的新好幫手：**Gemini CLI**！



Gemini CLI 是功能強大的指令列介面，可協助您：

- 工作流程自動化：有效管理 AI 專案。
- 生成程式碼：生成程式碼、UI，甚至是整個檔案。
- 擴充功能生態系統：使用各種擴充功能 (例如稍後會用到的 Cloud Run 擴充功能) 擴展功能。

想成為專業人士嗎？如要深入探索 Gemini CLI 的所有功能，請參閱 [Gemini CLI 動手練習程式碼實驗室](#)。您也可以參閱完整的 [Gemini CLI 說明文件](#)。

1. 安裝 Gemini CLI (僅限本機終端機)

如果使用本機筆電終端機：

注意：系統必須安裝 Node.js (v20 以上版本) 和 npm，才能使用 Gemini CLI。

```
npm install -g @google/gemini-cli
```

如果您使用 Google Cloud Shell：

Gemini CLI 已預先安裝在 Cloud Shell 中，因此您不必安裝任何項目！

2. 啟動並驗證

在 **Cloud Shell** (或本機終端機) 中開啟新的終端機視窗 (或分頁)，然後啟動 Gemini CLI：

```
gemin
```

首次執行設定：如果是第一次執行，CLI 會引導您完成快速設定程序。

- **驗證：**系統會要求你登入。建議使用個人 **Gmail 帳戶**，充分運用 Gemini 模型提供的[大量免費配額](#)。
- **專案環境：**建議您從專案資料夾內執行 `gemin`，這樣工具才能瞭解程式碼環境。(由於我們尚未建立資料夾，目前從主目錄執行即可)。

3. 打聲招呼！

看到 `gemin>` 提示後，請嘗試進行測試，確認大腦已連線：

```
Hi
```

模型應該會給出友善的回覆。

4. 基本指令

- `/quit`：輸入這個指令即可退出 Gemini CLI，返回標準終端機。
- `/help`：查看所有可用指令的清單。

專業提示：請在另一個分頁中開啟「Gemini 終端機」！您將使用這個終端機生成程式碼並部署應用程式，而其他終端機則會執行應用程式伺服器。

摘要

在這個步驟中，您已安裝、驗證及確認 Gemini CLI。

接著，我們將使用 Agent Development Kit (ADK) 建構商店的架構，請參閱「**Opening Shop (The Code)**」。

步驟 5 · 約 10 分鐘

5. 上架 (專案設定)

我們不會從頭開始建構 (聰明的店主會節省時間)。我們將使用 **Agent Development Kit (ADK)**。

什麼是 ADK？



ADK 是你的「商店管理系統」。這項彈性架構可處理下列事項：

- **自動化調度管理**：管理複雜工作 (例如 Raju 在與顧客通話時檢查庫存)。
- **狀態管理**：記住顧客的姓名和 5 分鐘前提出的要求。
- **工具**：連線至外部系統 (例如庫存資料庫)。

我們會使用 **Agent Starter Pack** 立即設定可供正式環境使用的「Dukaan」。這項服務提供安全且可擴充的基礎，讓我們專心銷售產品。

在 **Cloud Shell 終端機** (或本機終端機) 中執行下列指令，產生代理程式：

```
uvx agent-starter-pack create raju-shop
```

CLI 會詢問幾個問題，協助你設定商店。選擇下列選項 (輸入數字並按下 Enter 鍵)：

1. **代理程式類型**：選擇 **1** (`adk_base` - 基礎 ReAct 代理程式)。
2. **部署目標**：選擇 **2** (`Cloud Run` - 無伺服器容器執行)。
3. **工作階段類型**：選擇 **1** (`In-memory session` - 簡單又快速)。
4. **CI/CD Runner**：選擇 **1** (`Simple` - 我們目前會手動部署)。

等待程序完成。完成後，請輸入商店目錄並安裝依附元件：

```
cd raju-shop
make install
source .venv/bin/activate
```

探索及設定代理程式

設定好入門套件後，請檢查結構。

1. 檢查資料夾結構

執行下列指令，查看商店的組織方式：

```
tree .
```

您會看到類似以下的結構：

```
.
├── app
│   ├── __init__.py
│   ├── agent.py
│   ├── app_utils
│   │   ├── telemetry.py
│   │   └── typing.py
│   └── fast_api_app.py
├── Dockerfile
├── GEMINI.md
├── Makefile
├── pyproject.toml
├── README.md
├── tests
│   ├── integration
│   │   ├── test_agent.py
│   │   └── test_server_e2e.py
│   └── unit
│       └── test_dummy.py
└── uv.lock
```

最重要的檔案是 `app/agent.py`。這就是拉朱的大腦！

2. 逐步完成程式碼

在 [Cloud Shell 編輯器](#) (或本機 IDE) 中開啟 `app/agent.py`，並快速瀏覽：

- `root_agent = Agent(...)`：定義 AI。它有 `model` (大腦) 和 `tools` (雙手)。
- **工具**：您會看到 `get_weather` 等 Python 函式。這些是代理程式可使用的工具。
- `app = App(...)`：這會將代理程式包裝成網路伺服器，方便我們與其通訊。

想深入瞭解嗎？請參閱 [ADK Python 指南](#) 和 [LLM 代理說明文件](#)。

在本機測試

在 **Cloud Shell 終端機** (或本機終端機) 中執行代理程式：


```
adk web
```

這個指令會在通訊埠 **8000** 啟動本機網路伺服器。

如果使用 **Cloud Shell**：

1. 按一下「網頁預覽」按鈕 (位於右上角，外觀像眼睛)。
2. 選取「變更連接埠」。
3. 輸入 **8000**，然後按一下「變更並預覽」。

如果使用本機終端機：

1. 開啟瀏覽器並前往 **http://localhost:8000**。

開啟使用者介面後：

1. 選取「應用程式」：在左上方的下拉式選單中，請務必選取名為「應用程式」的代理程式 (如果顯示 **root_agent** 或 **test_agent**，請忽略)。
2. 說聲「Hello」：輸入「Hello!」，看看裝置是否會回應。

完成後，請在終端機中按下 **Ctrl+C**，停止 **adk web** 伺服器。

摘要

在本步驟中，您已搭建專案架構、為 Gemini API 設定專案，並成功在本機執行第一個代理程式！

接著，我們會在「Training Raju (The Persona)」中為代理程式設定個性與名稱。

步驟 6 · 約 10 分鐘

6. 訓練 Raju (角色)

目前這個代理程式很無趣，為他賦予個性！



我們不必編寫程式碼，只要告訴 Gemini 想要什麼即可。

1. 定義角色

在終端機中執行下列指令，讓 Raju 「認識」自己：

```
gemini "In agent.py, update the root_agent instruction. You are Raju, a bargaining shopkeeper in a digital bazaar. You sell: Brass Lamp (50 coins), Silk Scarf (500 coins). Your goal is to sell high and be funny. Speak with an Indian-English flair."
```

2. 驗證代碼

開啟 `app/agent.py`。您應該會看到 Gemini 為您撰寫的系統指令！

3. 在本機測試

現在，你可以和 Raju 對話了！在 **Cloud Shell 終端機** (或本機終端機) 中：

```
adk web
```

開啟網頁 UI (網頁預覽通訊埠 **8000** 或 **localhost:8000**)，選取 **app** 代理程式，然後嘗試下列提示：

- 「我想要銅製燈具，但 50 枚金幣太貴了！」
- 「你賣什麼，我的朋友？」

看看他的反應！他應該要有趣、戲劇化，但拒絕以太低的價格出售。

摘要

在這個步驟中，您自訂了代理程式的系統指令，建立具有獨特個性的「Raju」，也就是擅長討價還價的店主。
接著，我們會讓 Raju 能夠在「The Inventory (Adding Tools)」中查看實際庫存。

步驟 7 · 約 10 分鐘

7. 庫存 (新增工具)

Raju 需要知道實際的現貨數量。我們會提供「資料庫」和檢查工具給他。



1. 建立工具

執行這個單一指令，即可建立目錄、工具，並將工具連結至 Raju：

```
gemini "In agent.py, create a dictionary INVENTORY with items: Brass Lamp (price 50, stock 5), Silk Scarf (price 500, stock 2), Taj Mahal (price 2000, stock 0). Then create a tool function check_inventory(item_name) that checks this dict. Finally, update the root_agent to use this tool and remove the default weather tools."
```

2. 驗證 Magic

再次開啟「`app/agent.py`」，你會看到 Gemini 執行了下列操作：

1. 已建立 `INVENTORY` 字典。
2. 撰寫 `check_inventory` Python 函式。
3. 更新代理程式定義中的 `tools=[...]` 清單。

3. 測試廣告空間

1. 重新啟動伺服器 (按 `Ctrl+C`，然後按 `adk web`)。
2. 詢問 Raju：
 - 「你們有泰姬瑪哈陵的相片嗎？」(他應該說「否」，因為庫存為 0)。
 - 「絲巾多少錢？」(他應該查看實際價格)。

專業提示：在對話時查看終端機記錄。每次 Raju 使用工具時，您都應該會看到 `DEBUG: Checking inventory...` ！

摘要

在這個步驟中，您實作了 `check_inventory` 工具、移除了預設的雜亂內容，並將工具連結至代理程式。

接下來，我們將在「**Raju's Shop Goes Live (Deploying the Backend)**」中，將 Raju 的商店上線至雲端。

步驟 8 · 約 15 分鐘

8. Raju's Shop Goes Live (Deploying the Backend)

現在就向全球開放！在建構店面之前，我們會先將 Raju 的大腦和商品目錄 (你的代理程式) 部署到雲端。



重要事項：終端機管理

您可能已在先前步驟中，於目前的終端機執行 `adk web`。

- 如要繼續在本機測試，請保持執行。
- 開啟新的終端機視窗/分頁，以便執行後續部署步驟。
- 注意：每次變更 `agent.py` 時，都必須停止 (**Ctrl+C**) 並重新啟動 `adk web`，變更才會生效！

1. 準備使用 Gemini CLI

在新終端機視窗中：

1. 確認你是否已進入 Gemini CLI (尋找 `gemini>` 提示)。
2. 如果是，請輸入 `/quit` 退出至標準殼層。
3. 確認您位於專案資料夾中：

```
cd raju-shop
```

2. 安裝 Cloud Run 擴充功能

在 **Cloud Shell** 終端機 (或本機終端機) 中，為 Gemini CLI 安裝 **Cloud Run** 擴充功能：

```
gemini extensions install https://github.com/GoogleCloudPlatform/cloud-run-mcp
```

3. 重新啟動 Gemini CLI

再次啟動 Gemini CLI，即可使用新擴充功能：

```
gemini
```

您應該會看到 Gemini CLI 提示。

4. 部署代理程式

使用 Gemini CLI 擴充功能的方式有兩種：

選項 1：斜線指令 (手動)：您可以明確輸入指令和標記：

```
/deploy --source . --name raju-agent --region us-centrall --allow-unauthenticated
```

選項 2：自然語言 (建議)：這是更簡單且更強大的方式！只要要求 Gemini 執行這項操作，Gemini 就會使用 **MCP 伺服器** 呼叫適當的工具。

請嘗試將下列提示詞貼到 Gemini CLI：

檢查專案 ID：在下列提示中，將 `raju-shop-agent` 替換為實際的 **Google Cloud 專案 ID**。您可以在 Google Cloud 控制台的左上角下拉式選單中找到專案 ID，也可以在標準終端機中執行 `gcloud config get-value project`。

```
Deploy this agent app to cloud run on google cloud using the cloud-run MCP server.  
Use project raju-shop-agent in us-centrall region.  
Name the service raju-agent.  
IMPORTANT: Make sure to allow unauthenticated invocations so my frontend can talk to it.
```

CLI 可能會要求確認是否要使用 `cloud-run` 工具。按下 `Enter` 鍵 (或輸入 `y`) 即可核准。

等待 Magic 運作：這項作業需要 2 到 3 分鐘。完成後，您會看到類似下方的成功訊息：

```
✦ I've successfully deployed your agent app to Cloud Run.
```

Service Details:

```
* Service Name: raju-agent  
* Project: raju-shop-agent  
* Region: us-centrall  
* URL: https://raju-agent-xyz123-uc.a.run.app  
* Console: View in Google Cloud Console (...)
```

疑難排解：如果發生任何錯誤，只要以簡單的英文問問 Gemini CLI 錯誤即可！舉例來說：*"It failed with a permission error. 該如何修正？"*

5. 驗證部署

部署完成後，Gemini CLI 會提供「服務網址」。複製這個網址。

瀏覽器檢查 (是否正常運作?) 在瀏覽器中貼上網址。

- **成功：**您應該會看到 **FastAPI 說明文件 (Swagger UI)** 或一般 ADK 到達網頁。這可確認 Cloud Run 服務正在執行且可供存取。
- **失敗：**如果收到「403 Forbidden」錯誤訊息，可能是因為您忘記在部署提示中允許「未經驗證的呼叫」。

我們會在下一個步驟中，透過建構適當的前端來測試實際的議價邏輯！

摘要

在本步驟中，您使用 Gemini CLI 擴充功能將代理程式後端部署至 Google Cloud Run，並透過公開網址存取。

接下來，我們將在「**建構店面 (前端 UI)**」中，為顧客打造美觀的店面，方便他們與 Raju 對話。

步驟 9 · 約 10 分鐘

9. 裝飾商店 (前端 UI)

文字終端機很無聊。我們希望店面櫥窗充滿活力！



ADK API 的運作方式

如要建構自訂 UI，您必須瞭解如何與代理程式後端通訊。這項功能會依循以下 2 步驟流程：

1. **初始化工作階段：**開始對話前，請先建立工作階段 ID (例如購物車)。
 - `POST /apps/app/users/{user_id}/sessions/{session_id}`
2. **傳送訊息：**如要對話，請將文字傳送至執行端點。
 - `POST /run` (酬載包括 `appName`、`userId`、`sessionId` 和你的 `text`)。
3. **瞭解詳情：**請參閱 [ADK Runtime API](#) 和 [Python API 參考資料](#) 文件。

我們會使用這項邏輯建構前端。

您可以透過兩種方式建構這個介面：

方法 1：Google Antigravity (建議做法 - 僅限當地)

如果您是使用本機電腦工作，建議使用 [Google Antigravity](#)，這款 IDE 內建 AI 技術，可讓您「隨心所欲」編寫整個 UI 的程式碼。

附註：Antigravity 是電腦應用程式，如果您使用 **Cloud Shell**，請跳至選項 2。

1. 下載並安裝

前往 [Google Antigravity 下載頁面](#)，下載適用於您作業系統的安裝程式並安裝。

2. 開啟工作區

啟動 Antigravity。以 **Workspace** 身分開啟 `raju-shop` 資料夾。讓 AI 瞭解整個專案的背景資訊。

3. 使用 Agent Manager 生成 UI

我們將使用 **Agent Manager**，請 AI 代理程式代為編寫程式碼，不必逐行撰寫。

1. 找到 Agent Manager 即時通訊介面 (通常位於右側)。
2. 將下列提示詞貼入對話：

```
Create a single file HTML chat interface named `index.html`. It should have a colorful header that says 'Raju's Royal Artifacts'. It should look like an Indian market stall with orange and pink colors.
```

```
**Backend Integration:**
```

```
The chat should send messages to my deployed agent at: <YOUR_DEPLOYED_AGENT_URL>
```

```
**API Logic:**
```

1. ****On Load:**** Generate a random `userId` and `sessionId`. Call ``POST /apps/app/users/{userId}/sessions/{sessionId}`` to initialize.
2. ****On Send:**** Call ``POST /run`` with a JSON payload containing ``appName: "app"`, `userId`, `sessionId`, and `newMessage: { role: "user", parts: [{ text: userInput }]}`.`
3. ****Display:**** Show the user's message and the agent's response (from ``content.parts[0].text``).

(請務必將預留位置網址替換為部署步驟中的實際服務網址！)

1. 觀看代理為您撰寫程式碼！您可以切換至「編輯器檢視畫面」，即時查看檔案建立過程。

4. 測試店面

Google Antigravity 內建預覽功能，按一下「預覽」按鈕 (通常是眼睛圖示)，即可即時查看店面！或者，您也可以使用 `python3 -m http.server 8000` 在本機提供服務。

如要真正精通這個強大的 IDE，建議完成「[開始使用 Google Antigravity 程式碼研究室](#)」。

選項 2：Gemini CLI (與 Cloud Shell 相容)

如果您在 Cloud Shell 中，可以使用 Gemini CLI 在幾秒內建構前端。

在終端機中執行下列指令：

```
gemini "Create a single file HTML chat interface named index.html. Header: 'Raju\'s Royal Artifacts', Indian market theme (orange/pink). Backend: <YOUR_DEPLOYED_AGENT_URL>. Logic: On load, generate random user/session IDs and POST to /apps/app/users/{uid}/sessions/{sid}. On chat, POST to /run with appName='app', userId, sessionId, and newMessage structure. Parse the JSON response to show the agent text." > index.html
```

(請記得將前述文字中的預留位置 `<YOUR_DEPLOYED_AGENT_URL>` 換成實際的 Cloud Run 網址！)

摘要

在這個步驟中，您使用 Google Antigravity「直覺式程式開發」自訂 HTML 前端，並連結至即時服務專員後端。

接下來，我們將在「上線店面」中，在網路上託管這個店面，完成整個圖片。

步驟 10 · 約 10 分鐘

10. 盛大開幕 (部署前端)

Raju 在線上 (後端)，但他的商店需要公開地址 (前端)！要求 Google Antigravity 將 UI 部署至雲端。



1. 要求代理管理員部署

不必離開 IDE。只要請代理程式管理員為您處理部署作業即可。

將這則提示詞貼到對話中：

「Deploy my `index.html` as a static website to Google Cloud Run. 為服務命名 `raju-shop-frontend`。請確認該檔案可公開存取。」

2. 觀看魔術

服務專員可能會：

1. 建立 `Dockerfile` 或設定，用於提供靜態檔案 (例如使用 `nginx` 或 `Python`)。
2. 建構容器映像檔。
3. 為您執行 `gcloud run deploy` 指令。

3. 歡迎造訪商店！

完成後，代理程式會提供網址 (例如 `https://raju-shop-frontend-xyz.run.app`)，請點選該網址。

恭喜！您現在已完整部署 AI 驅動的討價還價店主應用程式。將連結傳送給朋友，看看他們是否能獲得折扣！

摘要

在這個步驟中，您已使用 Antigravity Agent Manager 將前端應用程式容器化，並部署至 Cloud Run。

您現在已建構完整堆疊！最後，我們會在「打烊時間 (結論)」中總結。

11. 結尾時間 (結論)



恭喜！您已順利完成「Bargaining Shopkeeper」代理程式挑戰！您將 Raju 從單純的想法，變成數位市集中的互動式 AI 店主，並完成部署。您已親身體驗代理程式開發的強大功能。

旅程回顧：

- **Google Cloud**：設定專案和帳單帳戶，並將代理程式部署至 **Cloud Run**。
- **Gemini 3**：為 Raju 的角色和議價能力提供智慧支援。
- **Agent Development Kit (ADK)**：這個架構提供可用於正式環境的基礎，並內建可觀測性 (遙測) 功能，讓您的代理從一開始就十分穩健。
- **Gemini CLI**：您的指令列助理，可快速完成工作、測試及擴充功能。
- **Google Antigravity**：新一代 IDE，您可以在其中「以程式碼打造 UI 氛圍」，並協調部署作業。

Digital Dukaan 後續步驟：

- **修正損毀的測試**：您變更了程式碼，但測試呢？預設測試 (`tests/unit/test_agent.py`) 仍會尋找天氣資訊！使用 Gemini CLI 自動更新。試試：`gemini "Update the tests in tests/unit/test_agent.py to test the new check_inventory tool instead of weather."`
- **動態商品目錄**：在商品「售出」時更新 Raju 的商品目錄數量。成功「購買」後，您可以在 `INVENTORY` 字典中新增邏輯，減少 `stock` 嗎？
- **Payment Gateway 工具**：實作模擬 `process_payment` 工具。這樣一來，Raju 就能在達成交易時呼叫這項工具！
- **探索 Antigravity**：深入瞭解 Antigravity 的代理程式管理員和工作區。請服務專員調整 `index.html`，顯示即時收據。
- **分享你的商店**：Raju 的商店已在網路上線！將前端網址分享給好友，看看他們是否能找到比你更划算的價格！

資源

- [Google Antigravity](#)
- [Google Cloud Run](#)

- [ADK 説明文件](#)
 - [Gemini CLI](#)
-

12. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的不必要費用，請按照下列步驟操作。

透過 **Google Cloud 免費方案** 繼續學習！

許多 Google Cloud 產品 (包括 Cloud Run) 提供**免費方案**，讓您在一定限度內免費使用資源。

如果您仍在 **Google Cloud 免費試用** 期內，還能享有更多彈性！

只要用量不超過免費方案限制，您就可以繼續使用目前的 Google Cloud 專案 (`raju-shop-agent`) 建構代理程式，不會遺失任何進度。

1. 取消部署 Cloud Run 服務 (可節省費用)

如要停止與已部署服務相關的任何潛在費用，同時保留專案和其他設定，可以取消部署 Cloud Run 服務。

開啟 **Cloud Shell 終端機** (或本機終端機)，然後執行下列 `gcloud` 指令：

```
# Undeploy the backend agent

gcloud run services delete raju-agent --region us-central1

# Undeploy the frontend shop (if you deployed it)

gcloud run services delete raju-shop-frontend --region us-central1
```

如果您是從本機電腦執行這些指令，請確認已安裝並設定 **Google Cloud CLI (gcloud)**。

2. 刪除 Google Cloud 專案 (完全停止計費)

如要確保不會再產生任何費用，並完全移除本程式碼研究室建立的所有資源，最明確的步驟就是刪除整個 Google Cloud 專案。

1. 前往 Google Cloud 控制台的「專案」頁面。
 2. 選取專案 (`raju-shop-agent`)，然後按一下「刪除」。
 3. 按照提示確認要刪除專案。
-